Database Architecture: PgBouncer + Replica + Redis Cache

Table of Contents

1. PgBouncer Workflow
2. Replica Setup Script
3. End-to-End API Flow
4. Architecture Diagram
5. Configuration
6. Verification

PgBouncer Workflow

Purpose

PgBouncer is a lightweight connection pooler that sits between the Node.js application and PostgreSQL: - **Reuses idle connections** — doesn't create a new one per request - **Multiplexes** many client connections over fewer server connections - **Reduces load** on PostgreSQL by managing connection lifecycle - Operates in **transaction mode** — releases connection after each transaction completes

Architecture

```
nodeapi-dev (app)
    │
    ↓ postgresql://postgres:postgres@pgbouncer:6432/devdb?pgbouncer=true
    │
PgBouncer (port 6432)
     │ connection pool
     │ (max 20 connections by default)
     │
    ↓ postgresql://postgres:postgres@postgresDB:5432/devdb
    │
PostgreSQL (port 5432)
    │
    ├──────────────────────────────────────┐
    │                                      │
    ▼                                      ▼
  Data                                  WAL Stream
  (tables, indexes)                    (replication logs)
                                              │
                                              ↓
                                        postgresRepDB (replica)
                                        port 5432 (read-only)
```

Connection Flow

1. Client Connection

```
nodeapi connects to pgbouncer:6432 with credentials: postgres/postgres
```

2. Authentication (PgBouncer → PostgreSQL)

PgBouncer uses `auth_query` to fetch real credentials:

```
SELECT usename, passwd FROM pg_shadow WHERE usename='postgres'
```

- Connects to PostgreSQL as the `auth_user` (postgres)
- Fetches SCRAM hash from `pg_shadow` system table
- Verifies client password against the hash

- If valid, routes client queries to PostgreSQL

3. Query Execution

```
Client query → PgBouncer → PostgreSQL
(INSERT/UPDATE/DELETE/SELECT)
```

4. Connection Release

After transaction completes:

```
Connection returned to pool (NOT closed)
Next client reuses same connection
```

Configuration (`pgbouncer.ini`)

```
[databases]
devdb  = host=postgresDB port=5432 dbname=devdb
testdb = host=postgresDB port=5432 dbname=testdb
demodb = host=postgresDB port=5432 dbname=demodb
proddb = host=postgresDB port=5432 dbname=proddb

[pgbouncer]
listen_addr         = 0.0.0.0
listen_port         = 6432
auth_type           = scram-sha-256
auth_file           = /etc/pgbouncer/userlist.txt
auth_user           = postgres
auth_query          = SELECT usename, passwd FROM pg_shadow WHERE usename=$1

pool_mode           = transaction
max_client_conn     = 1000
default_pool_size   = 20
min_pool_size       = 5
reserve_pool_size   = 5
```

Environment Control

```
# Enable PgBouncer
PGBOUNCER_ENABLED=true
PGBOUNCER_PORT=6432

# PRIMARY_DATABASE_URL routes through pgbouncer
PRIMARY_DATABASE_URL="postgresql://postgres:postgres@pgbouncer:6432/devdb?pgbouncer=true&connection_limit=1"
```

When `PGBOUNCER_ENABLED=false`, connection goes directly to `postgresDB:5432` (no pooling).

Replica Setup Script

Purpose

The replica initialization script (`scripts/replica-setup.sh`) handles: 1. **Database cloning** — Copy primary's data to replica 2. **Replication setup** — Configure streaming replication 3. **Permission fixes** — Handle Docker volume ownership

Script Execution

First Start (Empty Replica Data Directory)

```
#!/bin/bash

# 1. Fix permissions (volume mounted as root)
chown -R postgres:postgres $PGDATA
chmod 700 $PGDATA
```

```
# 2. Clone primary's data using pg_basebackup
pg_basebackup \
  -h postgresDB \
  -p 5432 \
  -U replicator \
  -D $PGDATA \
  -Fp -Xs -P -R

# 3. PostgreSQL creates:
#    - standy.signal  (marks this as a standby)
#    - recovery.signal + primary_conninfo (streaming replication config)

# 4. Start PostgreSQL in standby mode
exec gosu postgres postgres -c hot_standby=on
```

Options explained: - `-h postgresDB` — Primary host - `-U replicator` — Replication user (created in initdb) - `-D $PGDATA` — Destination directory - `-Fp` — Full backup in plain format - `-Xs` — Stream WAL (binary logs) during backup - `-P` — Progress reporting - `-R` — Write recovery configuration automatically

Subsequent Starts (Data Already Exists)

```
# Script skips pg_basebackup
echo "[Replica] Data directory exists, skipping pg_basebackup."

# Just start PostgreSQL in standby mode
exec gosu postgres postgres -c hot_standby=on
```

What Happens After Startup

```
PostgreSQL Replica Initialization Sequence:
│
├─ 1. Verify data directory is valid
│
├─ 2. Perform crash recovery (like normal PostgreSQL)
│     "database system was interrupted; last known up at..."
│
├─ 3. Enter standby mode
│     "entering standby mode"
│
├─ 4. Replay WAL (Write-Ahead Logs)
│     "redo starts at 0/2000028"
│     "completed backup recovery with redo LSN 0/2000028"
│
├─ 5. Reach consistent state
│     "consistent recovery state reached at 0/2000250"
│
├─ 6. Accept read-only connections
│     "database system is ready to accept read-only connections"
│
└─ 7. Start streaming WAL from primary
      "started streaming WAL from primary at 0/3000000 on timeline 1"
      Replica continually receives new changes via WAL stream
```

Replication User Setup

Created in `initdb/01_pgbouncer_user.sql`:

```
CREATE USER replicator WITH REPLICATION PASSWORD 'replicator';
```

Authenticated via `pg_hba.conf`:

```
host    replication     replicator      172.0.0.0/8             scram-sha-256
```

End-to-End API Flow

Scenario 1: Create User (POST /api/users)

```
┌─────────────────────────────────────────────────────────┐
│ Client Request                                          │
│ POST /api/users                                         │
│ { "name": "john", "email": "john@test.com" }            │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ nodeapi-dev (Express)                                   │
│ → POST handler → userController.createUser()            │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ userService.createUser()                                │
│ → primary_prisma.user.create({ data })                  │
│   (primary_prisma points to PRIMARY_DATABASE_URL)       │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ Prisma Client                                           │
│ prismaClient.js reads PGBOUNCER_ENABLED=true            │
│ Sets DATABASE_URL for primary:                          │
│ postgresql://postgres:postgres@pgbouncer:6432/...       │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ Network Request                                         │
│ nodeapi:5501 → pgbouncer:6432                           │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ PgBouncer Authentication                                │
│ 1. Client connects: postgres/postgres                   │
│ 2. PgBouncer runs auth_query:                           │
│    SELECT usename, passwd FROM pg_shadow                │
│    WHERE usename='postgres'                             │
│ 3. Fetches SCRAM hash from pg_shadow                    │
│ 4. Verifies password ✓                                  │
│ 5. Takes connection from pool (or creates new)          │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ PostgreSQL (Primary)                                    │
│ pgbouncer:6432 → postgresDB:5432                        │
│                                                         │
│ INSERT INTO User(name, email) VALUES(...)               │
│ → Writer process executes query                         │
│ → Updates data files                                    │
│ → Writes to WAL (Write-Ahead Log)                       │
│ → COMMIT                                                │
│                                                         │
│ ✓ Returns: id=42, name='john', email='j...'             │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ PgBouncer                                               │
│ Returns result to nodeapi                               │
│ Connection returned to pool (not closed)                │
└─────────────────────────────────────────────────────────┘
          │
          ▼
┌─────────────────────────────────────────────────────────┐
│ userService.createUser() continued                      │
│ await cacheAppend("users:all", 300, user)               │
│ → Redis: LPUSH users:all <user>                         │
│ → Sets TTL=300 seconds                                  │
└─────────────────────────────────────────────────────────┘
          │
          ▼
```

```
┌──────────────────────────────────────────────────────┐
│ userController                                       │
│ res.json(user)                                       │
└──────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────┐
│ HTTP Response                                        │
│ 201 Created                                          │
│ { id: 42, name: 'john', email: '...', ... }          │
└──────────────────────────────────────────────────────┘

┌──────────────────────────────────────────────────────┐
│ Replication (in parallel)                            │
│ PostgreSQL WAL file updated:                         │
│ PRIMARY sends new WAL segment to REPLICA             │
│ REPLICA applies INSERT to its copy                   │
│ Result: data available on replica immediately        │
└──────────────────────────────────────────────────────┘
```

Scenario 2: Get All Users (GET /api/users) - Cache HIT

```
┌──────────────────────────────────────────────────────┐
│ Client Request                                       │
│ GET /api/users                                       │
└──────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────┐
│ nodeapi-dev (Express)                                │
│ → GET handler → userController.getUsers()            │
└──────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────┐
│ userService.getUsers()                               │
│ return cacheGetOrSet(                                │
│   "users:all",                                       │
│   300,                                               │
│   () => replica_prisma.user.findMany()               │
│ )                                                    │
└──────────────────────────────────────────────────────┘
          │
          ▼
┌──────────────────────────────────────────────────────┐
│ cache.js cacheGetOrSet()                             │
│ 1. Try to GET "users:all" from Redis                 │
└──────────────────────────────────────────────────────┘
          │
     ┌────┴──────────────────────────┐
     │                               │
   CACHE HIT                     CACHE MISS
     │                               │
     ▼                               ▼
┌──────────────┐          ┌──────────────────────────┐
│ Redis        │          │ Call fallback fn:        │
│ has data     │          │ replica_prisma...        │
│              │          │   .findMany()            │
│ return       │          │                          │
│ [users...]   │          │ (see Scenario 3)         │
│ ✓ INSTANT    │          └──────────────────────────┘
└──────────────┘                     │
                                     ▼
                          ┌──────────────────────────┐
                          │ SELECT from              │
                          │ replica DB               │
                          │ Cache result             │
                          │ in Redis                 │
                          └──────────────────────────┘
                                     │
                                     ▼
                          ┌──────────────────────────┐
```

```
│ Return users to app │
└─────────────────────┘
        │
        ▼
┌─────────────────────┐
│ HTTP 200 OK         │
│ [                   │
│   { id: 1, ... },   │
│   { id: 42, ... }   │
│ ]                   │
└─────────────────────┘
```

Scenario 3: Get All Users (GET /api/users) - Cache MISS

```
┌──────────────────────────────────────────────────────┐
│ (Same as Scenario 2, but Redis cache empty)          │
│ → cacheGetOrSet() falls back to callback function    │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ replica_prisma.user.findMany()                       │
│ (replica_prisma points to REPLICA_DATABASE_URL)      │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ Prisma Client                                        │
│ prismaClient.js reads REPLICA_DATABASE_URL:          │
│ postgresql://postgres:postgres@postgresRepDB:5432/   │
│ (direct to replica, NO pgbouncer)                    │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ Network Request                                      │
│ nodeapi:5501 → postgresRepDB:5432                    │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ PostgreSQL Replica (Read-Only Standby)               │
│ 1. Verify: SELECT pg_is_in_recovery() → TRUE         │
│ 2. Accept connection as read-only                    │
│ 3. Execute: SELECT * FROM User                       │
│ 4. Return all rows                                   │
│                                                      │
│ Note: Data is streamed from primary via WAL          │
│       Replica is near-synchronous (microseconds)     │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ cache.js cacheGetOrSet()                             │
│ Received rows from replica                           │
│ Store in Redis:                                      │
│   SETEX "users:all" 300 <json>                       │
└──────────────────────────────────────────────────────┘
        │
        ▼
┌──────────────────────────────────────────────────────┐
│ Return users to app                                  │
│                                                      │
│ HTTP 200 OK                                          │
│ [ { id: 1, ... }, { id: 42, ... } ]                  │
│                                                      │
│ Next GET within 300s → Cache HIT (Scenario 2)        │
└──────────────────────────────────────────────────────┘
```

Scenario 4: Cache Invalidation (Update User)

```
PUT /api/users/42 { "name": "Jane" }
         │
```

```
          ▼
  userService.updateUser(42, { name: "Jane" })
          │
          ▼
  primary_prisma.user.update(...)  ← writes to primary via pgbouncer
          │
          ▼
  PostgreSQL: UPDATE User SET name='Jane' WHERE id=42
          │
          ├─ Updates data file
          ├─ Writes to WAL
          └─ Streams to replica
          │
          ▼
  cacheListUpdate("users:all", updated_user)
      └─ Updates cached user in "users:all" list
         (doesn't delete whole cache, just updates that user)
          │
          ▼
  cacheSet("user:42", updated_user)
      └─ Updates individual user cache
          │
          ▼
  HTTP 200 OK { id: 42, name: "Jane", ... }

  Next GET /api/users → Cache HIT with fresh data
```

Architecture Diagram

```
                          CLIENT
                            │
                ┌───────────┼──────────┐
                │           │          │
            POST/PUT       GET       DELETE
           (writes)    (reads, cached)  (deletes)
                │           │          │
                └───────────┼──────────┘
                            │
                         nodeapi-dev
                            │
        ┌───────────────────┼──────────────────┐
        │                   │                  │
      Redis              Prisma             Error
     (Cache)             Client             Handler
        │                  │  │               │
        │                ┌─┘  └─┐             │
        │                │      │             │
   Cache ops          Primary  Replica         │
   (append,           (writes) (reads)         │
    update,           DATABASE DATABASE        │
    remove)             URL      URL           │
        │                │        │            │
        │                ▼        ▼            │
        │             pgbouncer  Direct        │
        │             (pooling)  Connection    │
        │                │        │            │
        │                ▼        ▼            │
        │             PostgresDB PostgresRepDB │
        │             (primary)  (standby)     │
        │                │        │            │
        │            (accepts    (read-only)   │
        │             writes)                  │
        │                │                     │
        │                │                     │
        │                │                     │
        │             WAL Stream               │
        │            (replication)             │
        │                │                     │
        └────────────────┼─────────────────────┘
                         │
                (Continuous sync
                 via WAL logs)
```

Connection Routing Summary

```
┌──────────────────────────────────────────────────────┐
│ Request Type    │ Route                              │
├──────────────────────────────────────────────────────┤
│ INSERT/UPDATE   │ pgbouncer → PostgresDB (primary)   │
│ DELETE          │ pgbouncer → PostgresDB (primary)   │
│ SELECT (cached) │ Redis (instant, no DB query)       │
│ SELECT (miss)   │ Direct → PostgresRepDB (replica)   │
│ Cache ops       │ Redis client direct                │
└──────────────────────────────────────────────────────┘
```

Configuration

Environment Variables

All in `env_templates/.env.<env>`:

```
# Database
POSTGRES_DB_USER=postgres
POSTGRES_DB_PASSWORD=postgres
POSTGRES_DB=devdb  # Changes per env: testdb, demodb, proddb
POSTGRES_DB_PORT=5432
POSTGRES_REP_DB_PORT=5433  # Replica host port

# URLs
DATABASE_URL="postgresql://..."  # Legacy, not used by Prisma anymore
DIRECT_DATABASE_URL="..."        # For migrations (direct to primary)
PRIMARY_DATABASE_URL="..."       # App writes → pgbouncer
REPLICA_DATABASE_URL="..."       # App reads → replica

# PgBouncer
PGBOUNCER_ENABLED=true
PGBOUNCER_PORT=6432

# Redis
REDIS_ENABLED=false  # Set to true to enable caching
REDIS_HOST=app-redis
REDIS_PORT=6379
REDIS_TTL=300  # Cache lifetime in seconds
```

Docker Compose Profiles

```
# Start only primary + app (minimal)
docker compose --profile dev up

# Start primary + app + pgbouncer (pooling)
docker compose --profile dev --profile pgbouncer up

# Start primary + app + pgbouncer + replica (full HA)
docker compose --profile dev --profile pgbouncer --profile replica up

# Start primary + app + pgbouncer + replica + redis (full + caching)
# (Requires REDIS_ENABLED=true in .env)
docker compose --profile dev --profile pgbouncer --profile redis --profile replica up
```

Verification

1. Verify PgBouncer is Working

```
# Check pgbouncer logs
docker logs pgbouncer

# Expected output:
# LOG listening on 0.0.0.0:6432
# [when client connects]
```

```
# LOG C-0x...: devdb/postgres@... login attempt: db=devdb user=postgres
# S-0x...: new connection to server
```

2. Verify Replica Replication

```
# Check replica logs
docker logs PostgresRepDB

# Expected output:
# LOG entering standby mode
# LOG started streaming WAL from primary at 0/3000000 on timeline 1

# In replica PostgreSQL console
docker exec -it PostgresRepDB psql -U postgres -d devdb
devdb=# SELECT pg_is_in_recovery();
 pg_is_in_recovery
-------------------
 t
(1 row)
```

3. Verify Cache Working

```
# Test GET (should cache)
curl http://localhost:5501/api/users

# Check Redis has cached data
docker exec -it app-redis redis-cli
127.0.0.1:6379> KEYS "users:*"
1) "users:all"
127.0.0.1:6379> LLEN users:all
(integer) 2
```

4. Verify Read-Write Separation

```
# Create user (writes to primary via pgbouncer)
curl -X POST http://localhost:5501/api/users \
  -H "Content-Type: application/json" \
  -d '{"name":"test","email":"test@test.com"}'

# Get users (reads from replica, cached in Redis)
curl http://localhost:5501/api/users

# Check both pgbouncer AND replica logs to see activity
docker logs pgbouncer | grep "login attempt"
docker logs PostgresRepDB | tail -20
```

Troubleshooting

PgBouncer Connection Failed

```
Error: could not create connection pool: Error: expected 'd' at position 3 in ...
```

Solution: Check `PRIMARY_DATABASE_URL` format and verify pgbouncer is healthy:

```
docker logs pgbouncer
```

Replica Not Replicating

```
PostgresRepDB | LOG: could not connect to replication server
```

Solution: Verify replicator user exists and `pg_hba.conf` allows replication:

```
-- On primary
SELECT * FROM pg_stat_replication;
```

Cache Not Working

```
GET /api/users returns fresh data every time
```

Solution: Verify Redis is running and `REDIS_ENABLED=true`:

```
docker logs app-redis
redis-cli PING  # Should return PONG
```

Performance Notes

Read Performance

- **Cache HIT (90% of reads)**: < 1ms (Redis)
- **Cache MISS**: 10-50ms (replica query + cache store)

Write Performance

- **Primary write via pgbouncer**: 5-20ms (connection pooling overhead minimal)
- **Replica sync**: < 100ms (WAL streaming, near-synchronous)

Connection Pool Efficiency

- **Without pgbouncer**: 100 requests = 100 connections to PostgreSQL
- **With pgbouncer**: 100 requests = 5-10 connections (pooled)
- **Memory saved**: 85-95% reduction in PostgreSQL memory footprint

---

Production Checklist

- ☐ Set `PGBOUNCER_ENABLED=true` for all environments
- ☐ Set `REDIS_ENABLED=false` (enable only if needed)
- ☐ Verify replica replication in logs
- ☐ Monitor pgbouncer connection pool via `SHOW POOLS;`
- ☐ Set appropriate `REDIS_TTL` (300s default)
- ☐ Test failover: kill primary, verify app still works (writes fail, reads succeed from stale cache)
- ☐ Monitor WAL lag: `SELECT slot_name, restart_lsn FROM pg_replication_slots;`